

Product Requirements Document

Game Title: Perfect Stack

1. Overview

Perfect Stack is a simple, addictive browser game where the player stacks moving blocks as high as possible. The player taps, clicks, or presses the spacebar to place a moving block on top of the previous block. If the new block is not perfectly aligned, the overhanging part is cut off. The game continues until the player completely misses the stack.

The game must be built using only:

- HTML
- Vanilla JavaScript

No frameworks, libraries, build tools, canvas libraries, CSS frameworks, or game engines should be used.

2. Product Goal

Create a lightweight browser-based game that is extremely simple to understand, fast to load, and addictive through repeat attempts and high-score chasing.

The player should be able to start playing immediately without instructions longer than one sentence.

3. Target Users

The game is intended for casual players who want a quick game they can play during short breaks.

Primary users:

- Mobile users
 - Desktop users
 - Casual browser game players
 - Players who enjoy high-score challenges
-

4. Core Gameplay Concept

A block moves horizontally across the screen. The player must place it on top of the previous block by clicking, tapping, or pressing space.

If the block overlaps the previous block, the overlapping part remains and the non-overlapping part is removed.

If there is no overlap, the game ends.

The goal is to stack as many blocks as possible.

5. Core Game Loop

1. Player opens the game.
 2. Player sees the start screen.
 3. Player clicks, taps, or presses space to start.
 4. A block moves left and right.
 5. Player places the block.
 6. Game checks overlap with the previous block.
 7. If successful:
 8. Score increases by 1.
 9. Block size is trimmed if needed.
 10. New moving block appears.
 11. Speed gradually increases.
 12. If failed:
 13. Game over screen appears.
 14. Final score is shown.
 15. Best score is shown.
 16. Player can restart instantly.
-

6. Game Controls

Desktop

- Spacebar: Place block / Start / Restart
- Mouse click: Place block / Start / Restart

Mobile

- Screen tap: Place block / Start / Restart

The game should be playable with one hand on mobile.

7. Game Screens

7.1 Start Screen

The start screen should show:

- Game title: **Perfect Stack**
- Short instruction: **Tap or press space to stack**
- Best score, if available
- Start prompt

The first action from the player starts the game.

7.2 Gameplay Screen

The gameplay screen should show:

- Current score
- Best score
- Stack area
- Moving block
- Previous stacked blocks

The interface should remain minimal and distraction-free.

7.3 Game Over Screen

The game over screen should show:

- Text: **Game Over**
- Final score
- Best score
- Restart instruction: **Tap or press space to try again**

If the player beats the best score, show:

New Best Score!

8. Scoring Rules

- Player earns 1 point for every successfully placed block.
- Score starts at 0.
- The first base block does not count as a point.
- Each successful placement increases score by 1.

- Best score is stored locally in the browser using `localStorage`.
-

9. Difficulty Progression

The game should become harder over time.

Difficulty can increase through:

- Slightly increasing block movement speed after each successful placement.
- Reducing the available block width when the player places blocks inaccurately.
- Keeping the pace fast enough to encourage retries.

Suggested speed formula:

- Initial speed: 2 pixels per frame
 - Increase speed by 0.15 after every successful placement
 - Maximum speed: 8 pixels per frame
-

10. Block Placement Rules

Each block has:

- X position
- Y position
- Width
- Height
- Movement direction
- Movement speed

When the player places a block:

1. Compare the moving block with the block directly below it.
2. Calculate the overlap width.
3. If overlap width is less than or equal to 0:
4. End the game.
5. If overlap exists:
6. Keep only the overlapping section.
7. Remove the overhanging section.
8. Add the block to the stack.

9. Create the next moving block above it.

Example:

If the previous block starts at `x = 100` and has width `120`, and the new block starts at `x = 130` with width `120`, then the overlap is `90`.

The new block should become:

- `x = 130`
- `width = 90`

11. Perfect Placement Bonus

If the player places a block almost perfectly, the block should snap into perfect alignment.

A placement is considered perfect if the difference between the new block and previous block is very small.

Suggested perfect threshold:

- 5 pixels or less

When perfect placement happens:

- Align the new block exactly with the previous block.
- Keep the same width.
- Optional: show small text such as **Perfect!**
- Optional: give a subtle visual feedback.

This makes the game feel more satisfying and less punishing.

12. Visual Requirements

The game should have a clean and modern look.

Because the project should only use HTML and vanilla JavaScript, the recommended rendering method is:

- Use a single `<canvas>` element.
- Draw all game visuals using vanilla JavaScript.

Visual elements:

- Background
- Blocks

- Score text
- Start screen text
- Game over text

No external images are required.

13. Responsive Requirements

The game should work on:

- Desktop browsers
- Mobile browsers
- Tablet browsers

The canvas should automatically resize based on the browser window.

Recommended canvas behavior:

- Full-screen width
- Full-screen height
- Adjust game scale based on screen size

The game should remain playable on small mobile screens.

14. Audio Requirements

Audio is optional for version 1.

If added, use browser-native audio only.

Possible sounds:

- Block placed
- Perfect placement
- Game over
- New best score

Audio should not autoplay before the player interacts with the page.

15. Data Persistence

The game should store the best score locally.

Use:

```
localStorage
```

Required stored data:

- `perfectStackBestScore`

No backend is required.

No user account is required.

16. Technical Requirements

The project should contain one main HTML file.

Recommended file structure:

```
perfect-stack/  
└─ index.html
```

The `index.html` file should contain:

- HTML structure
- Canvas element
- Vanilla JavaScript game logic

No dependencies should be used.

Not allowed:

- React
- Vue
- Angular
- jQuery
- Phaser
- Three.js
- Tailwind
- Bootstrap
- NPM packages
- Build tools
- TypeScript
- External APIs

Allowed:

- HTML
 - Canvas API
 - Vanilla JavaScript
 - Browser events
 - `requestAnimationFrame`
 - `localStorage`
-

17. Main Game States

The game should have the following states:

`start`

The game has not started yet.

`playing`

The player is actively stacking blocks.

`gameOver`

The player missed the stack and the game has ended.

18. Main JavaScript Components

Game State

Responsible for tracking:

- Current screen
- Score
- Best score
- Blocks
- Current moving block
- Speed
- Canvas size

Input Handler

Responsible for:

- Click events

- Tap events
- Spacebar events

Game Loop

Responsible for:

- Updating moving block position
- Drawing the game
- Running continuously using `requestAnimationFrame`

Collision / Overlap Logic

Responsible for:

- Comparing the current block with the previous block
- Calculating overlap
- Determining success or failure

Renderer

Responsible for drawing:

- Background
- Blocks
- Score
- Start screen
- Game over screen

19. User Stories

User Story 1: Start Game

As a player, I want to start the game with one tap or key press so that I can play immediately.

Acceptance criteria:

- Player can start by clicking.
- Player can start by tapping.
- Player can start by pressing spacebar.
- The game begins immediately.

User Story 2: Place Block

As a player, I want to place a moving block on top of the stack so that I can build higher.

Acceptance criteria:

- Moving block stops when the player acts.
 - Game calculates overlap.
 - Score increases if placement is valid.
 - Game ends if placement misses completely.
-

User Story 3: Trim Block

As a player, I want inaccurate placements to reduce the block size so that the game becomes harder.

Acceptance criteria:

- Overhanging section is removed.
 - Remaining block matches the overlap area.
 - Next block uses the new trimmed width.
-

User Story 4: Track Score

As a player, I want to see my current score so that I know how well I am doing.

Acceptance criteria:

- Score is visible during gameplay.
 - Score increases by 1 after each successful placement.
-

User Story 5: Save Best Score

As a player, I want my best score saved so that I can try to beat it later.

Acceptance criteria:

- Best score is saved in localStorage.
 - Best score remains after page refresh.
 - New best score updates immediately after game over.
-

User Story 6: Restart Quickly

As a player, I want to restart quickly after losing so that I can try again immediately.

Acceptance criteria:

- Tap, click, or spacebar restarts the game.
 - Restart does not require refreshing the page.
-

20. MVP Scope

The MVP should include:

- Start screen
 - Canvas-based game area
 - Moving block
 - Stack placement
 - Overlap detection
 - Block trimming
 - Game over condition
 - Current score
 - Best score
 - Restart behavior
 - Mobile and desktop input support
 - LocalStorage high score
-

21. Out of Scope for MVP

The following should not be included in the first version:

- Multiplayer
 - User login
 - Online leaderboard
 - Backend server
 - Power-ups
 - Skins
 - Ads
 - In-app purchases
 - External images
 - External libraries
 - Complex animations
 - Level system
-

22. Future Enhancements

Possible future improvements:

- Online leaderboard
 - Daily challenge mode
 - Color themes
 - Combo system for perfect placements
 - Sound effects
 - Haptic vibration on mobile
 - Share score button
 - Different block shapes
 - Unlockable backgrounds
 - Progressive achievements
-

23. Success Metrics

The game should be considered successful if:

- A new player understands the game within 3 seconds.
 - The first game can be started with one action.
 - Restarting after game over is instant.
 - The game works on both desktop and mobile.
 - The game has no external dependencies.
 - The player is encouraged to retry after losing.
-

24. Acceptance Criteria

The finished version must meet the following requirements:

- Runs by opening `index.html` in a browser.
 - Uses only HTML and vanilla JavaScript.
 - Does not require installation.
 - Does not require internet access.
 - Does not use external libraries.
 - Works on desktop and mobile.
 - Supports click, tap, and spacebar.
 - Stores best score using `localStorage`.
 - Has clear start, playing, and game over states.
 - Allows instant restart.
 - Has smooth animation using `requestAnimationFrame`.
-

25. Recommended MVP Development Order

1. Create `index.html`.
 2. Add a full-screen canvas.
 3. Set up canvas resizing.
 4. Create game state.
 5. Draw the base block.
 6. Add moving block.
 7. Add player input.
 8. Implement block placement.
 9. Implement overlap detection.
 10. Implement block trimming.
 11. Add scoring.
 12. Add game over state.
 13. Add restart behavior.
 14. Add best score with `localStorage`.
 15. Polish visuals and mobile responsiveness.
-

26. One-Sentence Game Instruction

Tap or press space when the block lines up. Don't miss.